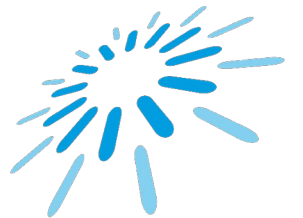




Software Architecture and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 09



TH Aschaffenburg
university of applied sciences

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 09:30					Vorlesung Software Design
09:45 - 11:15					Übung Software Design
11:30 - 13:00			Lecture Software Design International		
	Lunch Break				
14:00 - 15:30			Exercises Software Design International		
15:45 - 17:15					
17:30 - 19:00					

Next Week

Final Presentations for Bonus Points

In the lecture slot

What is a layered architecture?

How does a layered architecture affect the people working with it?

Distributed and Web Architectures



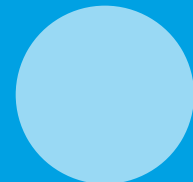
Intro - Modern Web Architectures by Example

Distributed Architectures

Service-Oriented Architectures

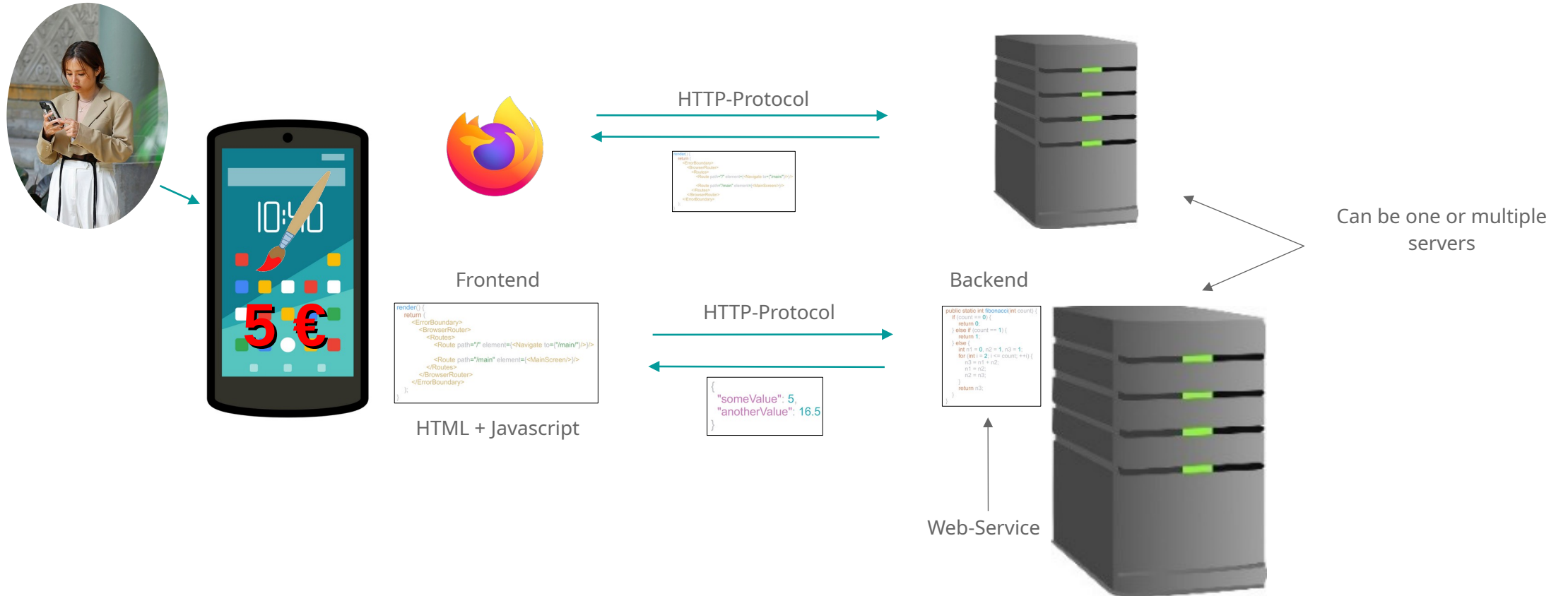
Microservice-Architectures

Cloud Computing



Hint:
Some interactive examples
today require an eduVPN
connection!

Modern Web Applications 101



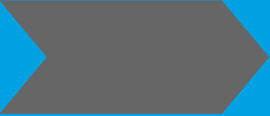
The Cloud

- Basic principle
 - Renting of software and hardware resources
 - Pay per use
- Important for now:
 - The ability to rent (virtual) servers
 - Payment per second/minute

Instance name ▲	On-Demand hourly rate ▼	vCPU ▼	Memory ▼
c7a.medium	\$0.05857	1	2 GiB
c7a.large	\$0.11713	2	4 GiB
c7a.xlarge	\$0.23426	4	8 GiB
c7a.2xlarge	\$0.46852	8	16 GiB
c7a.4xlarge	\$0.93704	16	32 GiB
c7a.8xlarge	\$1.87408	32	64 GiB
c7a.12xlarge	\$2.81112	48	96 GiB
c7a.16xlarge	\$3.74816	64	128 GiB
c7a.24xlarge	\$5.62224	96	192 GiB
c7a.32xlarge	\$7.49632	128	256 GiB
c7a.48xlarge	\$11.24448	192	384 GiB
c7a.metal-48xl	\$11.24448	192	384 GiB
c7g.medium	\$0.0412	1	2 GiB
c7g.large	\$0.0825	2	4 GiB
c7g.xlarge	\$0.1649	4	8 GiB
c7g.2xlarge	\$0.3298	8	16 GiB
c7g.4xlarge	\$0.6596	16	32 GiB
c7g.8xlarge	\$1.3192	32	64 GiB
c7g.12xlarge	\$1.9788	48	96 GiB
c7g.16xlarge	\$2.6384	64	128 GiB

Distributed and Web Architectures

Intro - Modern Web Architectures by Example

- 
- 1. Basics and Recap
 - 2. Horizontal Scalability
 - 3. Architectural Patterns
 - 4. Scaling at Runtime

Distributed Architectures

Service-Oriented Architectures

Microservice-Architectures

Cloud Computing

Example Application

`https://swa.sd.lv.th-ab.de/tax/`

Requires eduVPN connection!

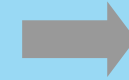
Web Site is slow under Load

What can we do?
(without touching the code)



We rent a server!

But which one?

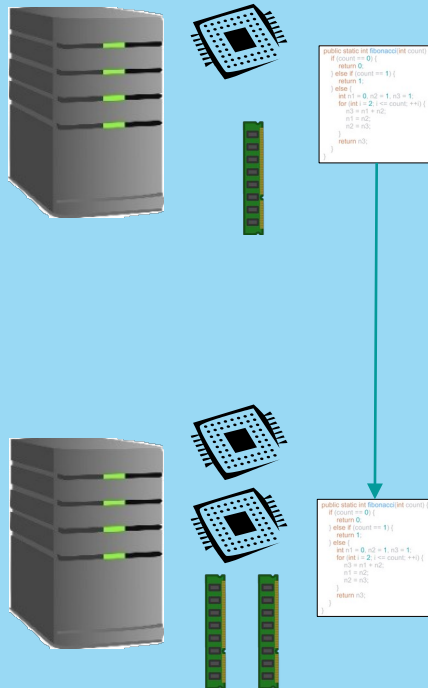


Instance name ▲	On-Demand hourly rate ▼	vCPU ▼	Memory ▼
c7a.medium	\$0.05857	1	2 GiB
c7a.large	\$0.11713	2	4 GiB
c7a.xlarge	\$0.23426	4	8 GiB
c7a.2xlarge	\$0.46852	8	16 GiB
c7a.4xlarge	\$0.93704	16	32 GiB
c7a.8xlarge	\$1.87408	32	64 GiB
c7a.12xlarge	\$2.81112	48	96 GiB
c7a.16xlarge	\$3.74816	64	128 GiB
c7a.24xlarge	\$5.62224	96	192 GiB
c7a.32xlarge	\$7.49632	128	256 GiB
c7a.48xlarge	\$11.24448	192	384 GiB
c7a.metal-48xl	\$11.24448	192	384 GiB
c7g.medium	\$0.0412	1	2 GiB
c7g.large	\$0.0825	2	4 GiB
c7g.xlarge	\$0.1649	4	8 GiB
c7g.2xlarge	\$0.3298	8	16 GiB
c7g.4xlarge	\$0.6596	16	32 GiB
c7g.8xlarge	\$1.3192	32	64 GiB
c7g.12xlarge	\$1.9788	48	96 GiB
c7g.16xlarge	\$2.6384	64	128 GiB

Idea 1

„Vertical Scaling“

More resources for our server!
(or migrate to a bigger server)



Advantages

- High confidence that our code will still work
- Lots of software benefits automatically *

Disadvantages

- Usually downtime when adding or removing resources
- Potentially expensive

Scalability

Definition:

“Scalability [...] connotes the ability to accommodate an increasing number of elements or objects, process growing volumes of work gracefully, and/or to be susceptible to enlargement.”

e.g. increasing load



The diagram features a central definition of scalability in a white box. Three arrows point towards this box from different directions: one from the bottom-left, one from the bottom, and one from the bottom-right. Each arrow is accompanied by a text label representing an example of a scalability factor.

e.g. more CPUs or servers

e.g. growing dataset sizes

Idea 2

„Horizontal Scaling“

Let's run our software on multiple servers!



Advantages

- Fine granular cost control
- High availability

Disadvantages

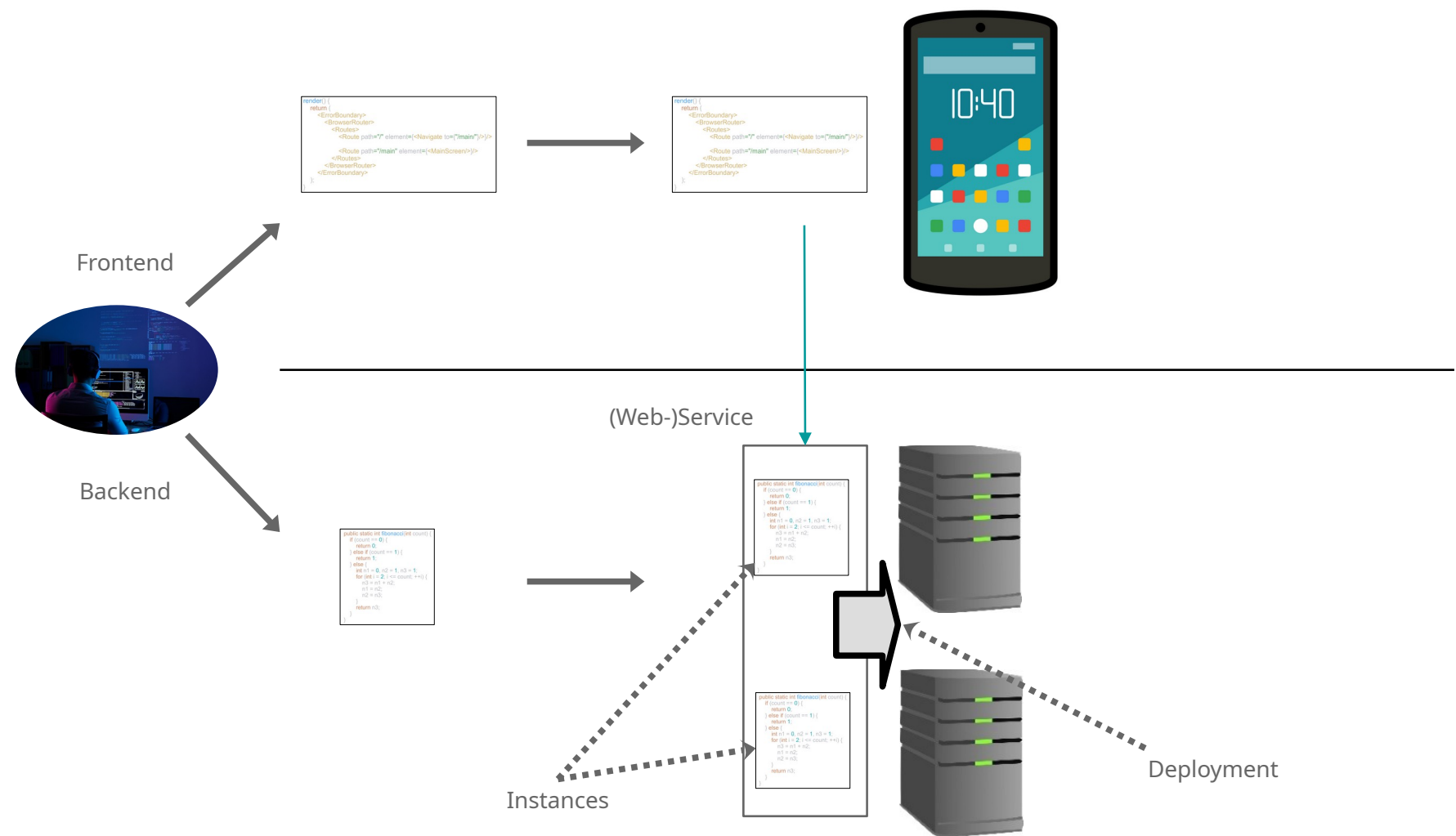
- Requests must be split
- Software architecture must support this

Software that doesn't scale horizontally

`https://swa.sd.lv.th-ab.de/click/`

Requires eduVPN connection!

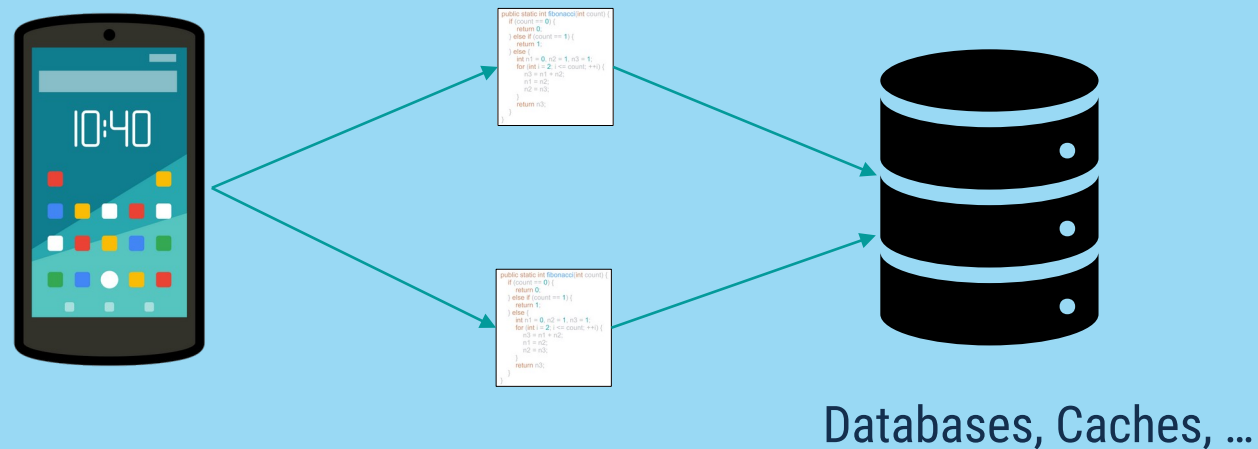
Terms



Ideal Properties of horizontally scaling Web-Services

- No state
- No instance identity

How do we deal with wanting to store data?



Distributed and Web Architectures

Intro - Modern Web Architectures by Example

- 
- 1. Basics and Recap
 - 2. Horizontal Scalability
 - 3. Architectural Patterns
 - 4. Scaling at Runtime

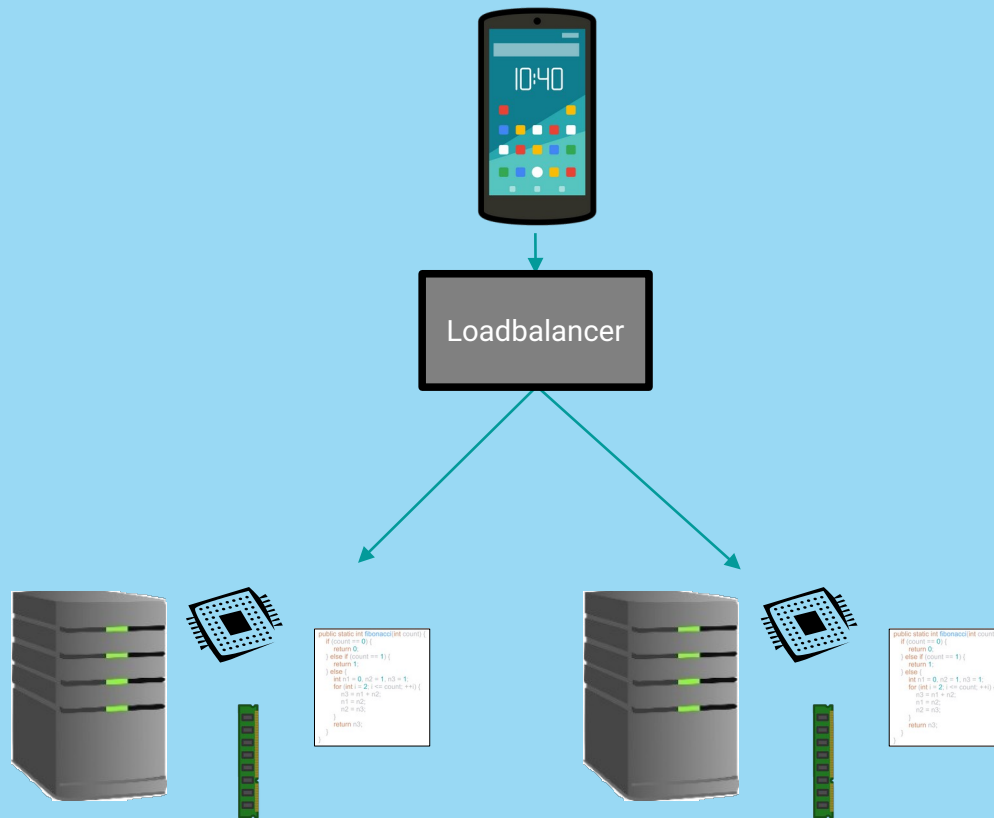
Distributed Architectures

Service-Oriented Architectures

Microservice-Architectures

Cloud Computing

Loadbalancer



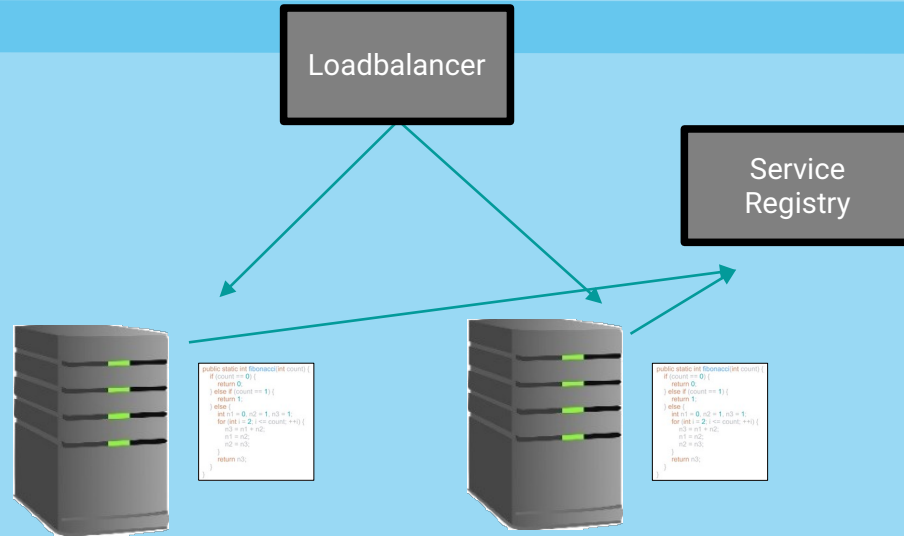
Problem

- Requests must be distributed across instances

Solution

- Loadbalancer accepts requests centrally and distributes them
 - May be hardware device, software component, or cloud service
 - Often a reverse proxy module
- Distribution typically round-robin

Service Discovery / Service Registry



Solution Alternative 1: Platform Service-Registry

- Platform (e.g., cloud) knows, which instances exist and where they are deployed
 - Requirement: Application meta information must be provided to platform
- Platform configures load balancer
- Platform provides interface

Problem

- Loadbalancer must know which instances exist
- (In certain forms of service-2-service communication: other services must know which instances exist)

Solution Alternative 2: Application Service-Registry

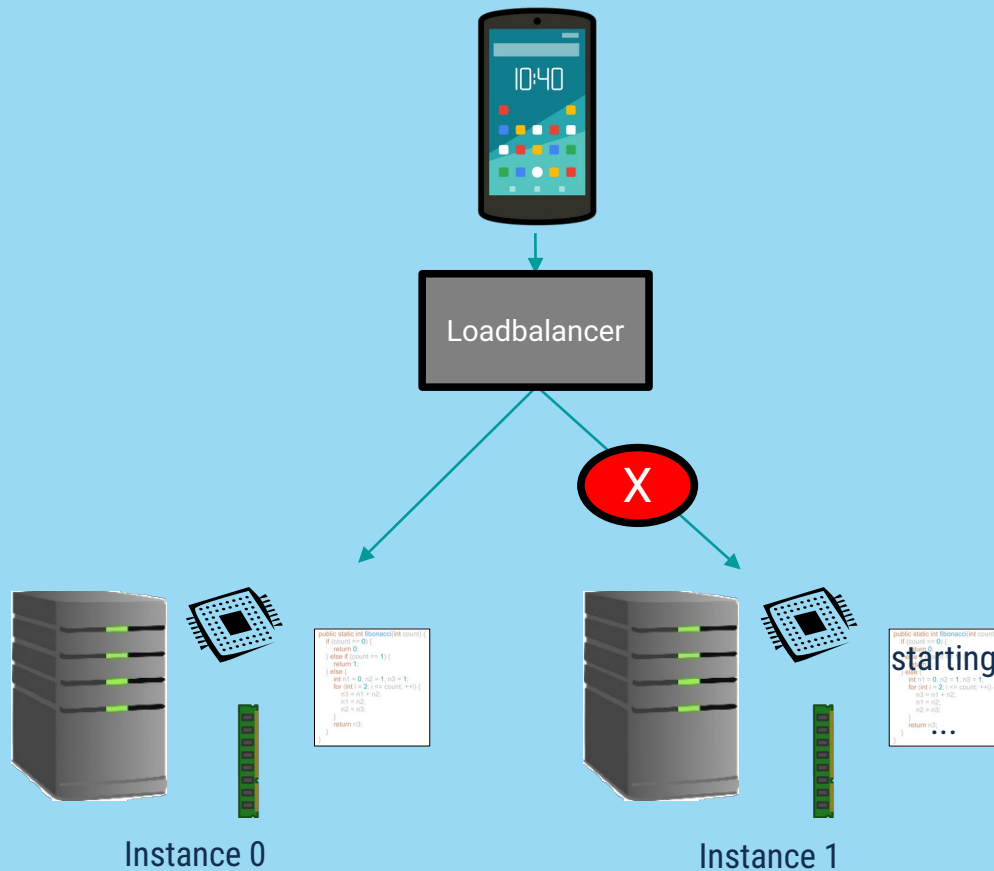
- Additional, separately deployed service
- Instances must actively register and de-register
- Service registry provides API

Scaling live!

`https://swa.sd.lv.th-ab.de/tax/`

Requires eduVPN connection!

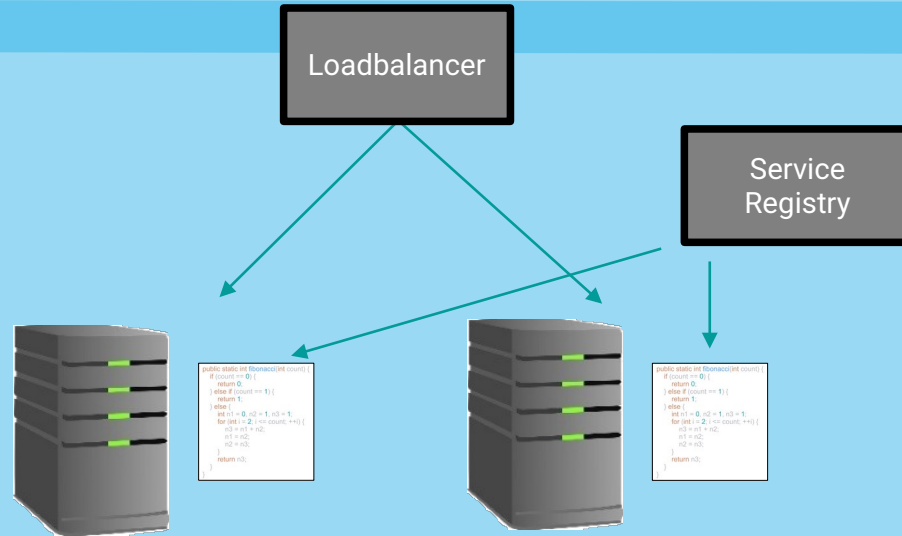
What happened?



Step by step

- 1) Load arrives at instance
- 2) Additional instance is started
- 3) Instance is registered at platform load balancer
- 4) Application on instance 1 starts up
- 5) Requests are distributed across instances, even though the application on instance 1 has not finished starting
- 6) Instance 1 is ready
- 7) All requests are successful

Health Check



Optional: Details

- Health interface may provide details about state of required services and/or hardware
 - Is the database available?
 - Is storage space available on disk?

Problem

- Server availability != Application availability
- Service registry must know if application is available

Solution

- Instances provide network API to check their state ("health")
- Service registry or platform calls this API regularly
- Instances only count as "UP" if the API returns OK

Distributed and Web Architectures

Intro - Modern Web Architectures by Example

- 
- 1. Basics and Recap
 - 2. Horizontal Scalability
 - 3. Architectural Patterns
 - 4. Scaling at Runtime

Distributed Architectures

Service-Oriented Architectures

Microservice-Architectures

Cloud Computing

Autoscaling



~~< 1s~~ ~~2s~~



TaxCalculator

TaxCalculator
#0

TaxCalculator
#1

Objectives

- Meet performance objectives
- Minimize costs
- Fully automated

Definition:

“the degree to which a system is able to adapt to workload changes [...] in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible”

- Describes how well horizontally scaling systems can meet their quality (performance) objectives at runtime, while also minimizing costs

Autoscaling in the Real World (or: Why is our elasticity not perfect?)

Rule-based autoscaler as part of the platform

- Monitors server metrics
- Makes scaling decisions based on metric thresholds

Advantages

- No implementation required by developers
- Good platform integration

Disadvantages

- Configurable metrics do not correlate perfectly with performance and costs
→ difficult to implement

× Manage Autoscaling

✓ Success: Scaling Rules saved successfully ×

[ENABLE AUTOSCALING](#) [Download Autoscaler CLI](#)

INSTANCE LIMITS

Currently in use: 1

Minimum 2 Maximum 3 [APPLY CHANGES](#)

SCALING RULES [EDIT](#)

Metric	Low	High
CPU Utilization	20%	80%

SCHEDULED LIMITS [EDIT](#)

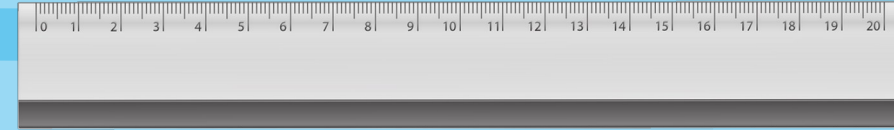
No Scheduled Limit Changes

EVENT HISTORY

No Events

[VIEW MORE](#)

Concluding Part 1



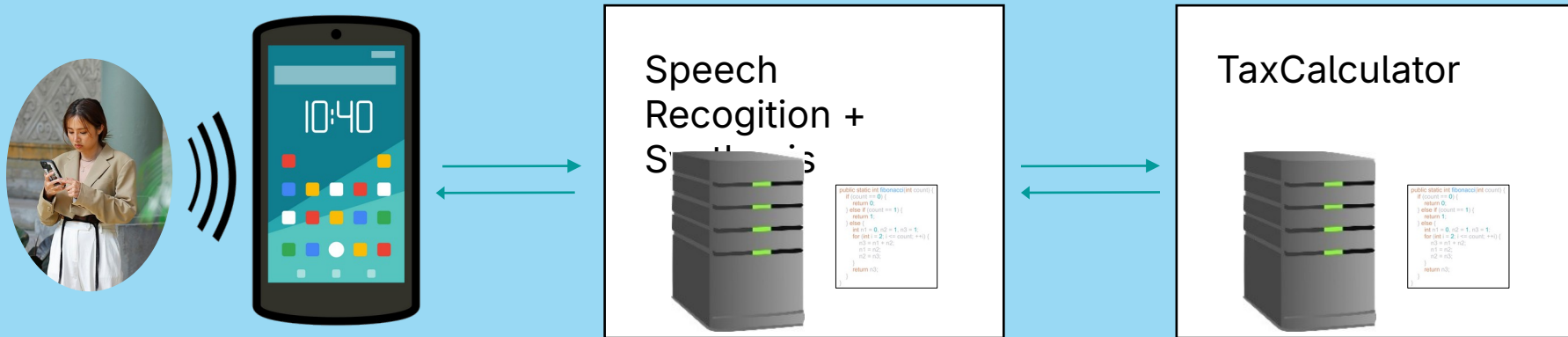
Scalability ...

**“the ability to [...] process growing volumes of work gracefully,
and/or to be susceptible to enlargement.”**

- **Horizontal scalability:** Enlargement by adding further instances
- **Some Architectural patterns for horizontally scaling systems:**
 - Loadbalancer, Service registry, Health check, Autoscaling
- **Elasticity:** How fast and how fitting is the exact amount of required resources provided?

Next Steps

Let's build some voice control!



- **Distributed Architecture with Microservices!**

Distributed and Web Architectures



Intro - Modern Web Architectures by Example

Distributed Architectures

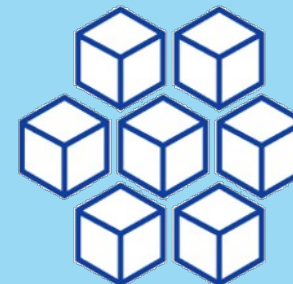
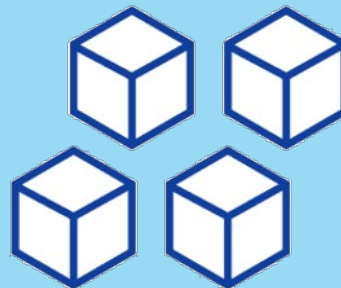
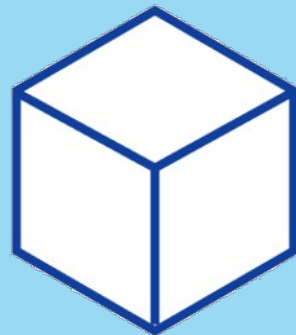
Service-Oriented Architectures

Microservice-Architectures

Cloud Computing

Monolithic vs. Distributed Architectures

- **Monolithic:** The whole system is bundled into one deployable unit
 - **Pro:** Simple architecture
 - **Con:** Single point of failure, limited room for scaling
- **Distributed:** The system is deployed into multiple (independent) units communicating over a network
 - **Pro:** Scalability, elasticity, robustness, and availability
 - **Con:** More complex and challenging problems than monolithic architectures
 - **Con:** Heavily dependent on the offered reliability, latency, bandwidth, cost, security, and topology of the underlying network



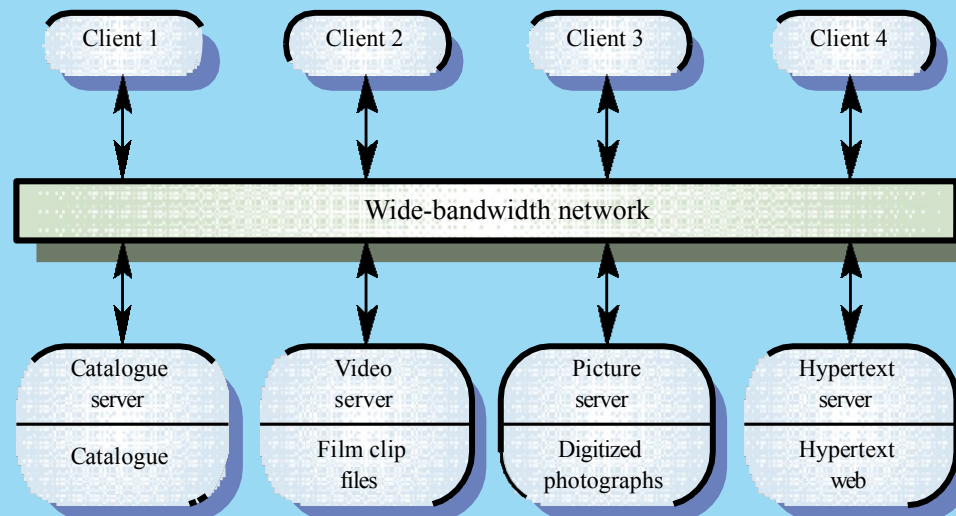
(Classic) Distributed Architectures

- Client-server architectures
 - Structuring of a system that partitions tasks or workloads between the providers of a resource or service, called **servers**, and service requesters, called **clients**
- Multi-tier architectures
 - Structuring of a system into **tiers** that implement distinct features such as the presentation logic or the application logic
 - Clear separation of responsibilities between tiers (typically include client-server relationships)
- Peer-to-peer architectures
 - System consists of interconnected **peers** of similar capabilities and responsibilities, where the peers can act as both servers and clients
 - Designed to be self-organizing and to scale well with the number of participating peers
- Distributed object architectures (e.g., CORBA, RMI)
 - any object on the system may provide and use services from other objects
 - no distinction between client and server objects

(Classic) Client-Server Architecture (C/S)

Distributed system model showing how data and processing are distributed across a range of computing resources

- A set of stand-alone **servers** that provide specific services such as printing, data management, etc.
- A set of **clients** that call upon these services
- A **network** that allows clients to access servers



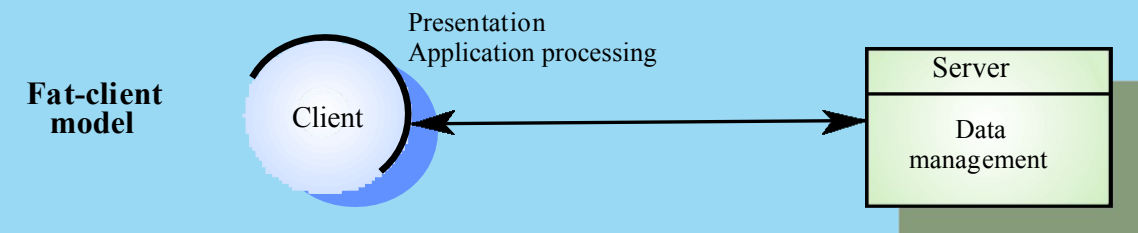
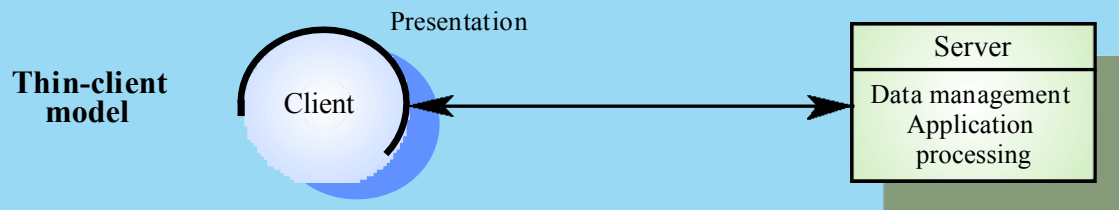
(Classic) Thin- and Fat-Client Architecture

Thin-client model

- **Application processing** and **data management** is carried out on the server
- Client is simply responsible for data **presentation / user interface (UI)**
- Often used when legacy systems are migrated to client-server architectures
- Places a heavy processing load on the server (and the network)

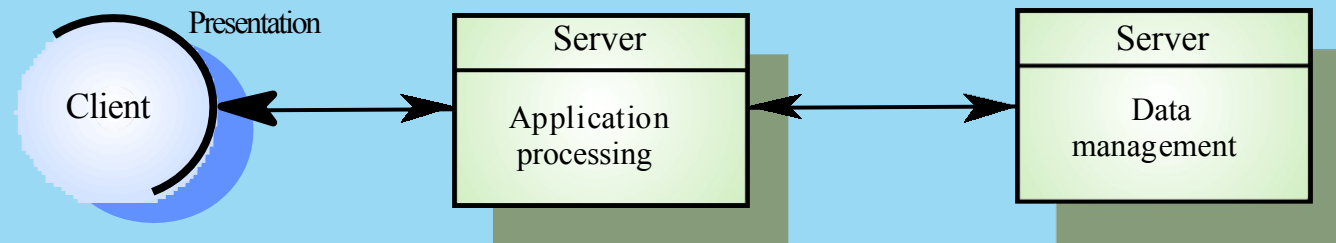
Fat-client model

- The server is only responsible for **data management**
- Client software implements **application logic** and **presentation / user interface (UI)**



(Classic) Multi-Tier Architecture

- In the context of enterprise information systems (EIS), the classic 3-tier deployment architecture has gained high importance
- Each architectural tier of an application executes on a separate machine
 - **Tiers** thus refer to physical nodes



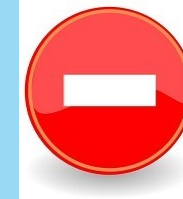
- Even **4-tier architectures** are common today
 - Separating the server on the left into a **web server** (for presentation) and an **application server** (for processing)
- Preferred for large complex software systems requiring performance and scalability
 - As performance demands increase, servers can be added where needed

Distributed Systems: Pros and Cons (On paper, missing the most important aspect)



Advantages

- ✓ resource sharing
- ✓ openness
- ✓ concurrency
- ✓ scalability
- ✓ fault tolerance
- ✓ transparency



Disadvantages

- complexity
- security
- manageability
- unpredictability
- testability

Question

**Regarding Conway's law:
How might the presented classical distributed
architectures influence people and organizations?**

Distributed and Web Architectures

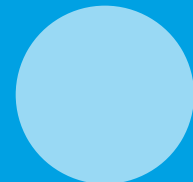
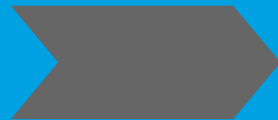
Intro - Modern Web Architectures by Example

Distributed Architectures

Service-Oriented Architectures

Microservice-Architectures

Cloud Computing



Service-Oriented Architecture (SOA)

Service-oriented architecture is a way of designing, developing, deploying, and managing systems, in which

- Services provide **reusable business functionality**,
- Service consumers are built using functionality from available services,
- Service **interface definitions** are first-class artifacts,
- SOA infrastructure enables **discovery, composition, and invocation** of services,
- Protocols are predominantly, but not exclusively, **message-based document exchanges**.

Component Paradigm vs. Service Paradigm

Component-based Software Engineering (CBSE)

- Main focus: How to design and compose software systems from **independently developed building blocks** (software components provided by third parties) without understanding their internals
- More focus on design time

Service-oriented Architecture (SOA)

- Main focus: How to compose software systems from **independently operated services** (deployed components) without having knowledge and understanding of their implementation, deployment, and operation details
- Weak coupling for maximum interoperability
- Focus on both design and run time (operation)

Services

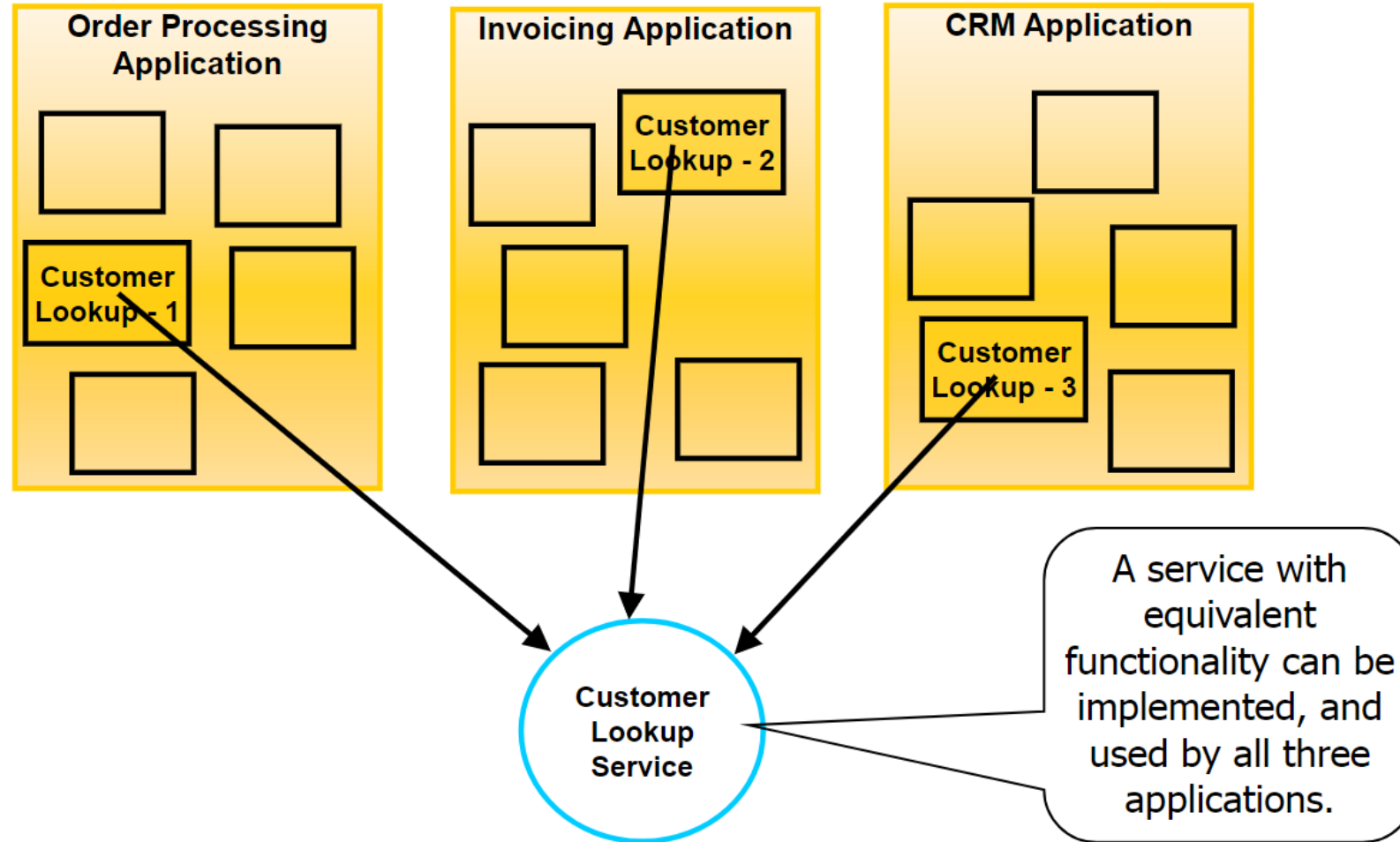
- Services are reusable components that represent business tasks
 - Customer lookup
 - Credit card validation
 - Weather
 - Hotel reservation
- Services can be
 - Globally distributed across organizations
 - Reconfigured into new business processes
- Service interface definitions are well-defined first-class artifacts (ideally) available in a service repository



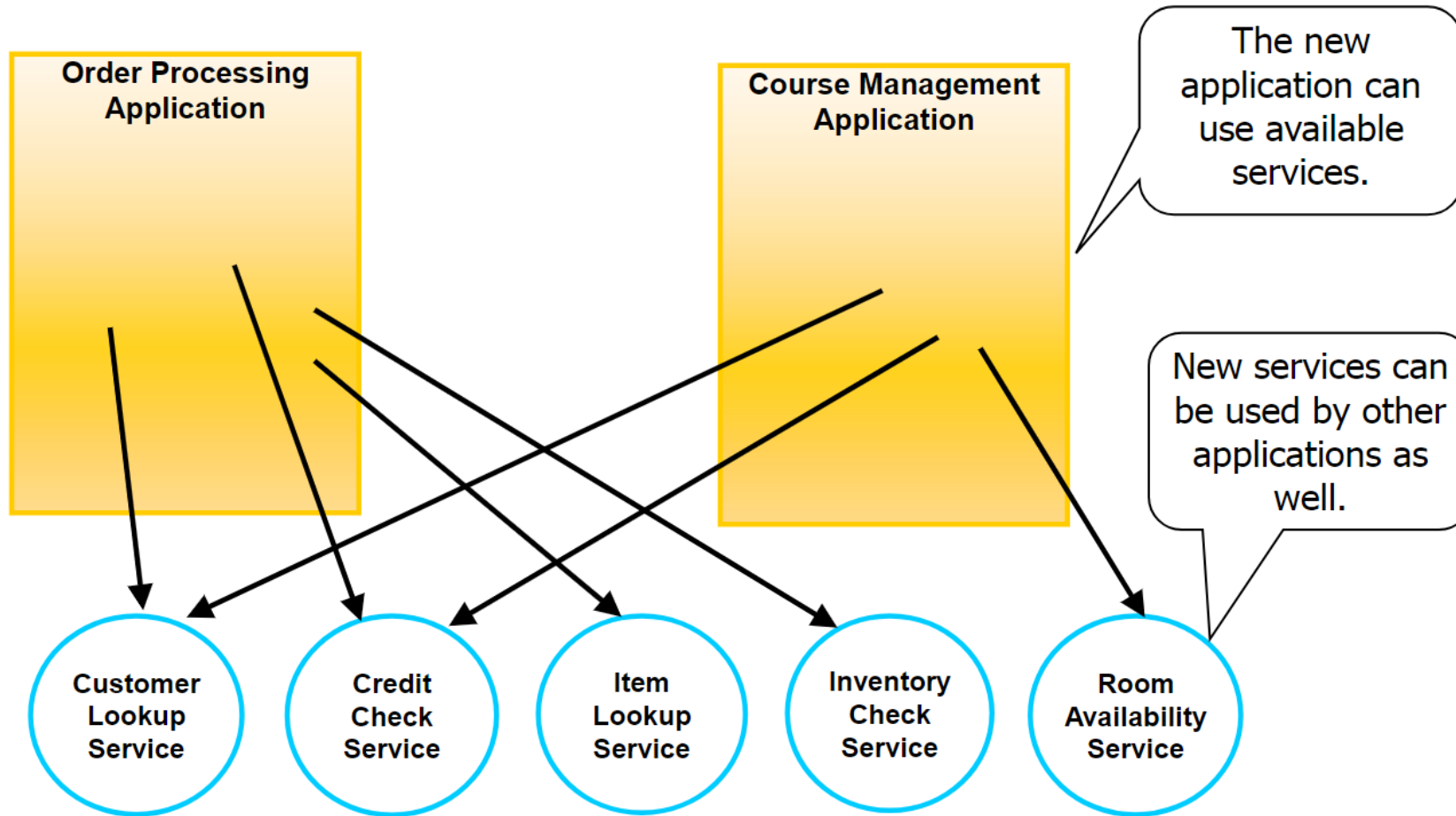
Service Consumers

- Clients for the functionality provided by the services
 - End-user applications
 - Internal systems
 - External systems
 - Composite services
- Consumers programmatically bind to services

Services and Cost-Efficiency

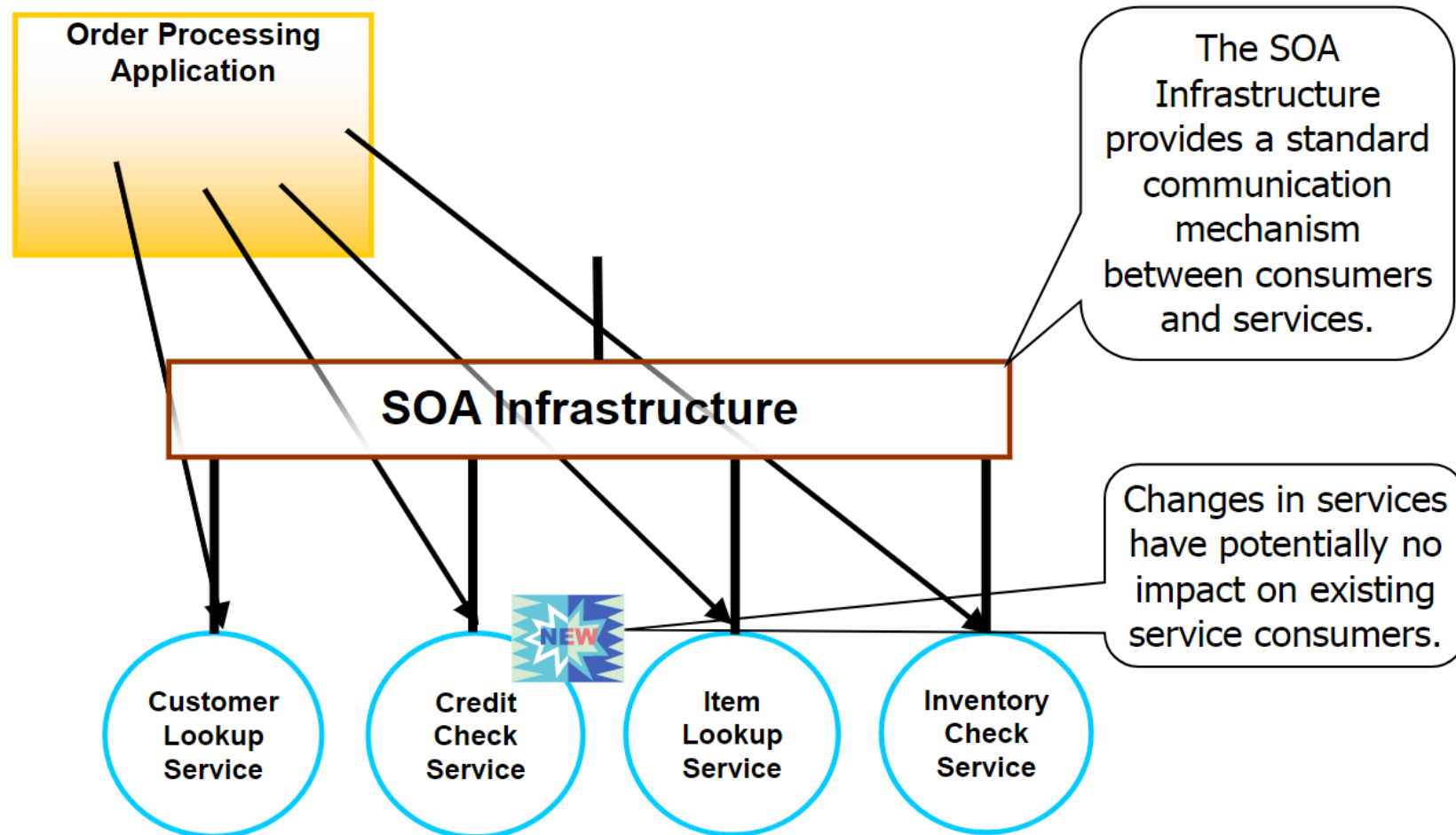


Services and Agility

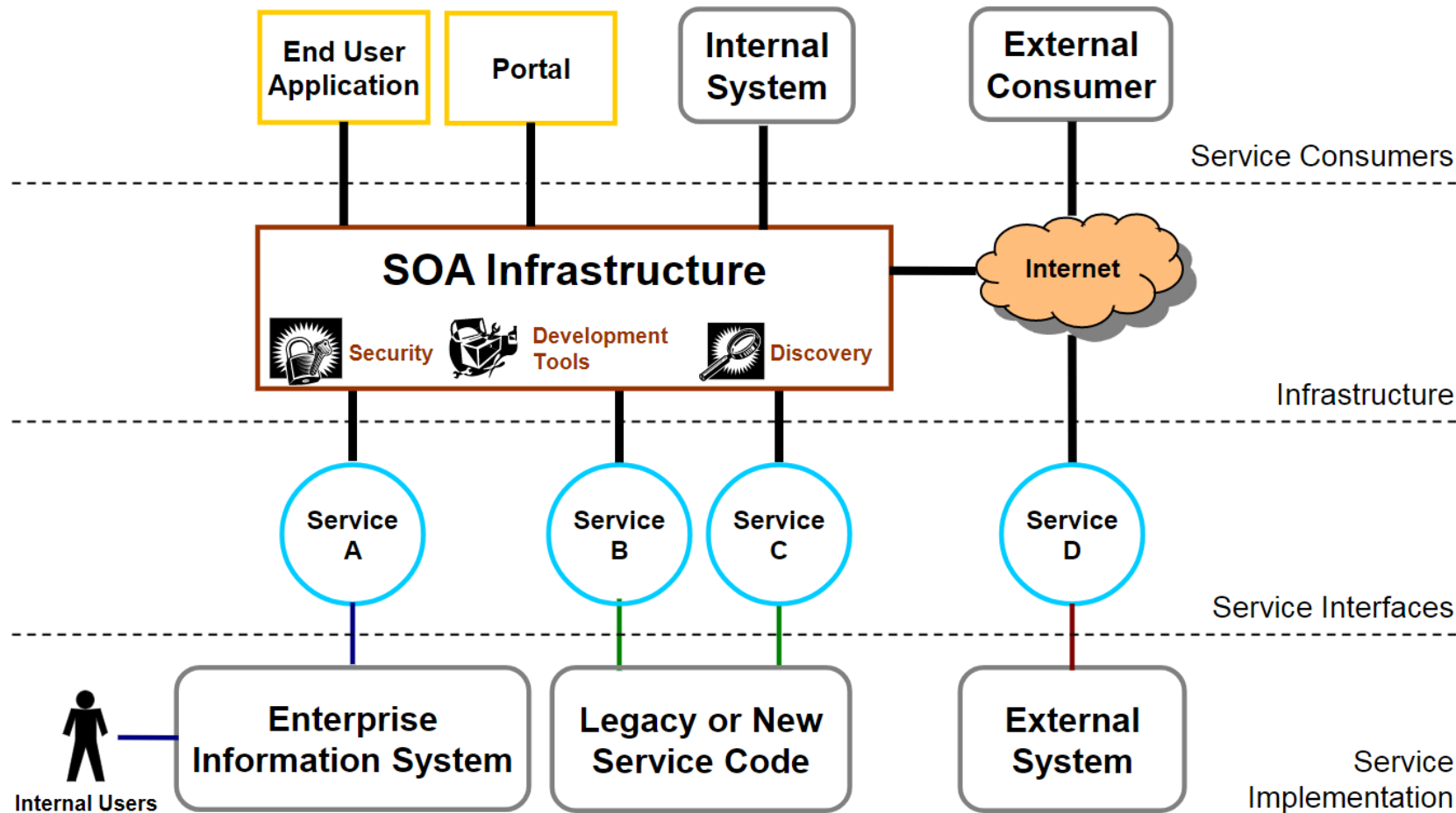


**Regarding Conway's law:
How can the adoption of services increase an
organizations speed and agility?**

Services and Adaptability



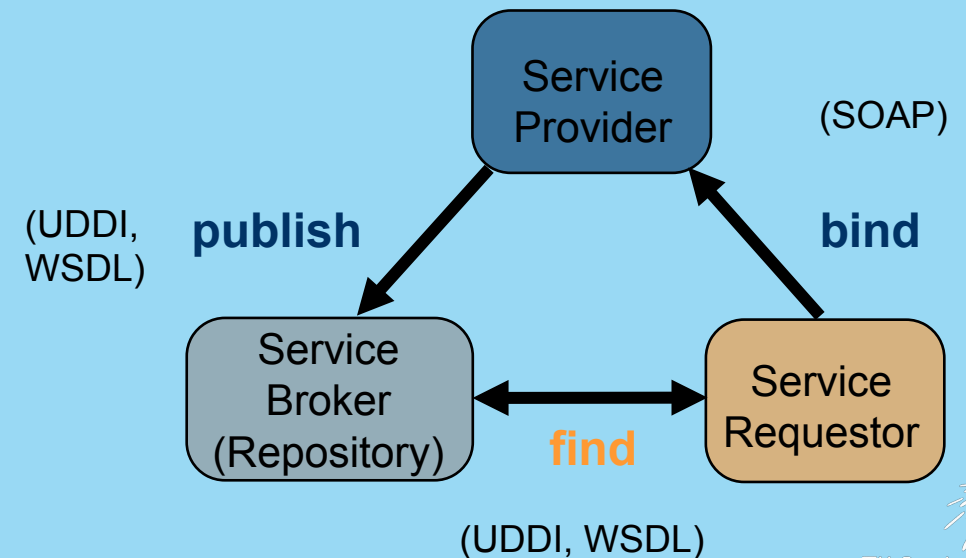
Components of a SOA System



WS* Web Services

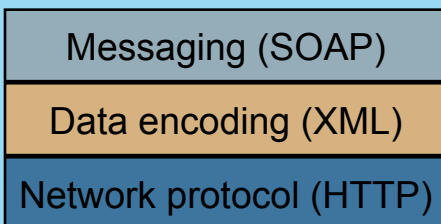
Web Services is one mechanism for implementing a service-oriented system

- Service interfaces are described using Web Services Description Language (WSDL)
- Data is transmitted using Simple Object Access Protocol (SOAP) over HTTP
- Universal Description, Discovery and Integration (UDDI) is optionally used as the directory service
- Consumers programmatically bind to services

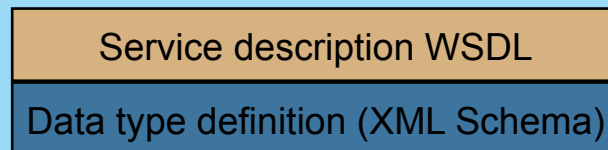


Core Web Service Technologies

- **SOAP** (Simple Object Access Protocol)
 - a message layout specification defining a uniform way of passing XML-encoded data
 - a way to simulate remote procedure calls (RPCs) over standard Web communication protocols
- **WSDL** (Web Service Description Language)
 - defines Web Services as collections of network endpoints or ports
 - a port is defined by associating a network address with a binding
- **UDDI** (Universal Description, Discovery and Integration)
 - provides a mechanism for clients to find web services
 - the basis for repository services for business applications



Interaction Stack



Description Stack



Discovery Stack



Distributed and Web Architectures

Intro - Modern Web Architectures by Example

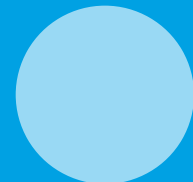
Distributed Architectures

Service-Oriented Architectures



Microservice-Architectures

Cloud Computing



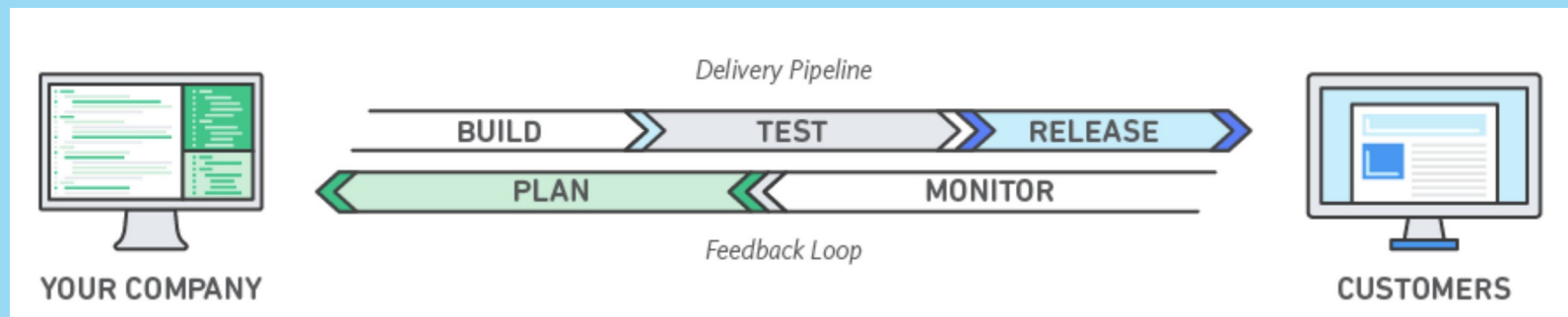
The Product Team – Idea

DevOps!

Goal: Single Team should be possible to deliver customer value all on its own!

To this end:

- Development, operations, and quality assurance
 - No longer strictly separated teams
- **Automate as much as possible**
 - Deployment, operations, and quality assurance
- **Developers deploy and provision** infrastructure
- **Rapid release cycle with a quick feedback loop**



Question

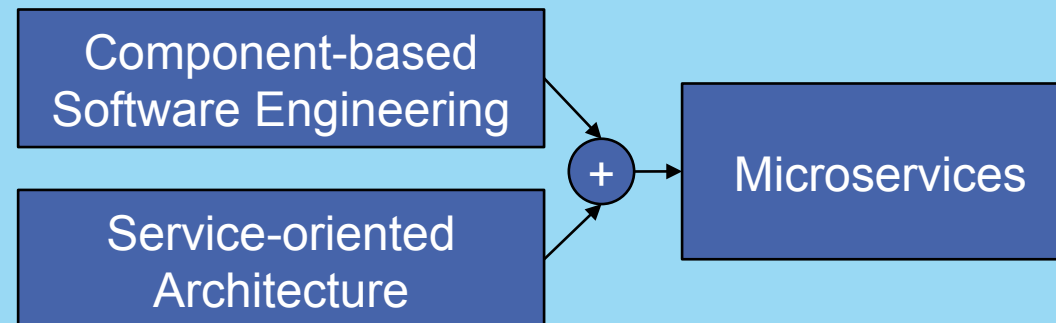
Why is “Single Team should be possible to deliver customer value all on its own!” a good idea?

Question

Why is “Single Team should be possible to deliver customer value all on its own!” usually not the case?

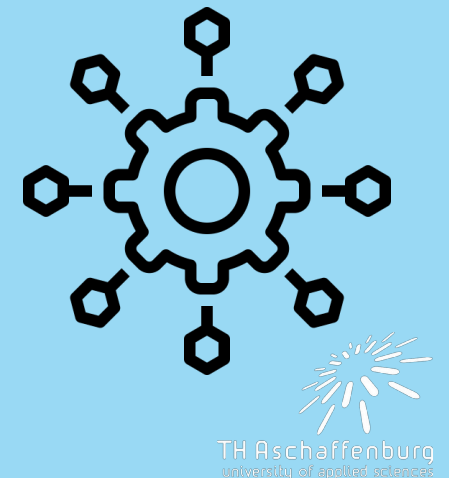
Microservices

- Microservice architectures aim to combine the benefits of both the CBSE and SOA paradigms, while enabling Product Teams
 - **CBSE**: Build software systems from **independently developed** components without understanding their internals
 - **SOA**: Deploy and operate components independently as **services** that can be used by others without having knowledge of their implementation, deployment, and operation details
- Weak coupling for maximum flexibility and interoperability
- Simplified view:

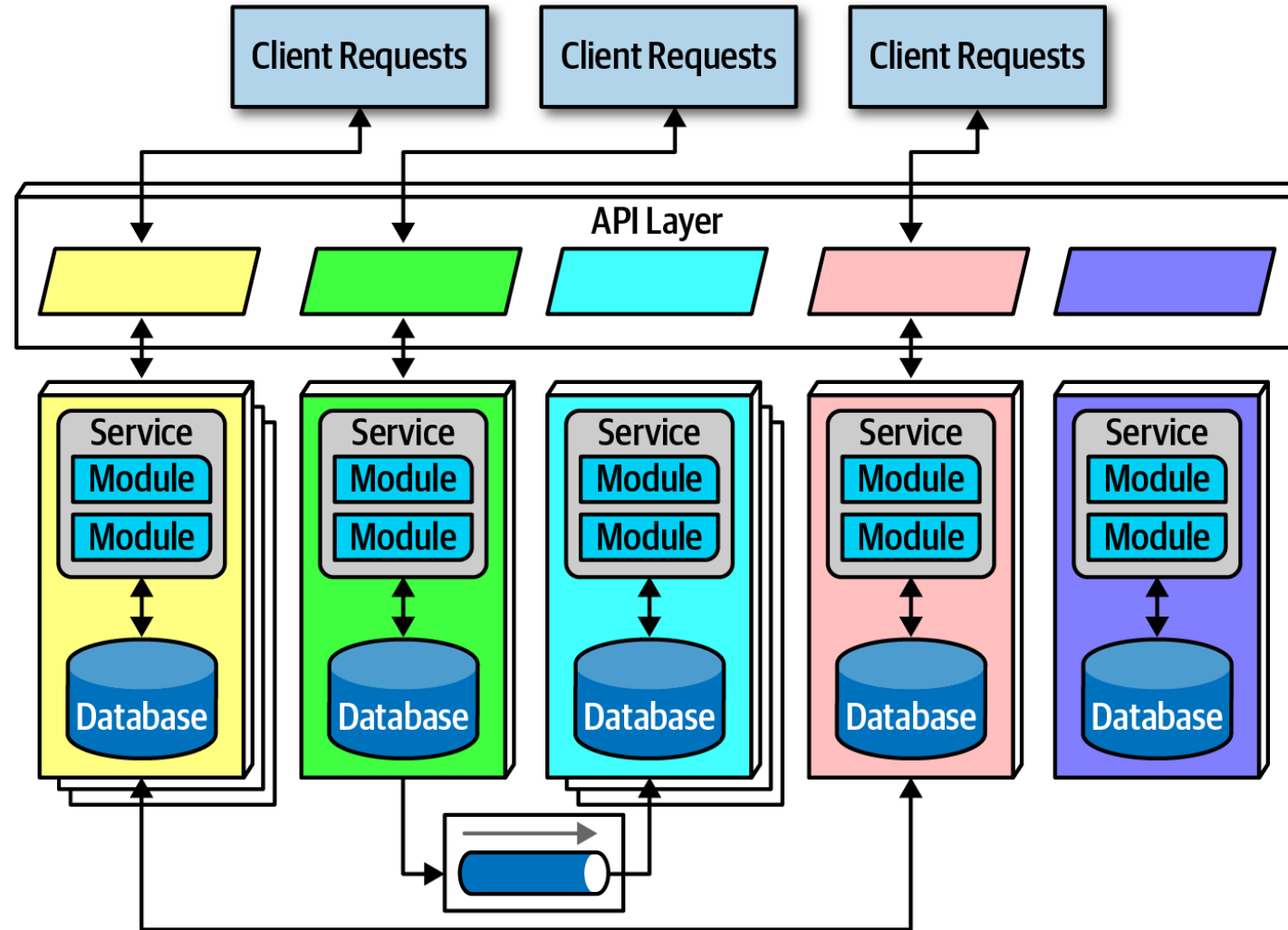


Microservice Architecture

- Microservice applications are composed of a set of **independent, distributed, and context-bounded** services
- Each service only has a fine-grained **single purpose**, leading to small and manageable units (hence the name **microservice**)
- The services can be **deployed and scaled independently**, and they **communicate** with each other over **network** (e.g., using REST)
- Usually, each service is **responsible** of managing its **own data**

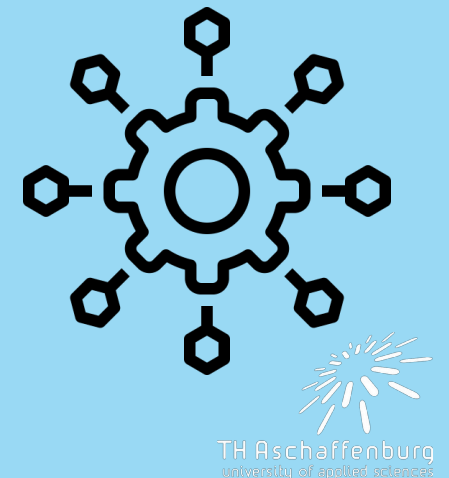


Microservice Architecture



Microservice Development

- Microservices development is commonly coupled with **modern software development** paradigms like DevOps, continuous integration, continuous deployment, containerized deployment, ...
- A separate **development team** might be responsible of the whole **lifecycle** (developing, testing, deploying, maintaining, documenting,...) of a given microservice
- Also enables the choice of **different technologies** (e.g., programming languages, middleware, databases) for each microservice → Polyglot
- Can also be coupled with an **agile** development process

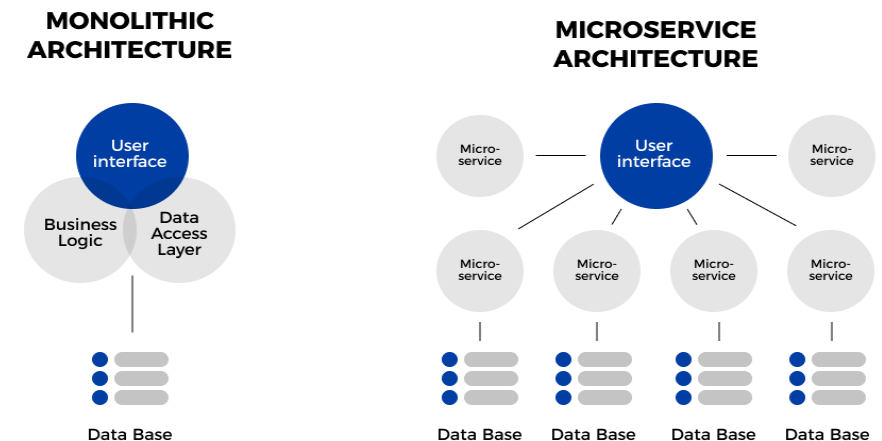


Microservice Architecture

Definition:

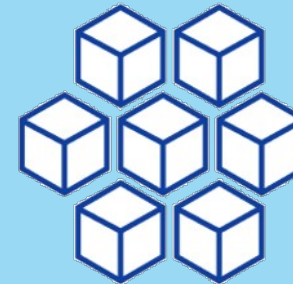
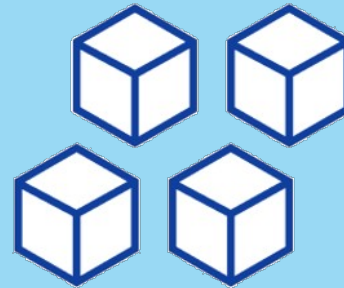
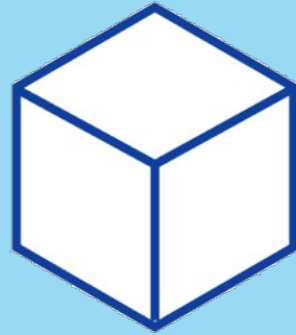
*“In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and **communicating with lightweight mechanisms**, often an HTTP resource API. These services are built around business capabilities and **independently deployable by fully automated deployment** machinery. There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies”.*

– Microservices Guide, James Lewis and Martin Fowler (2014)



Source: <https://divante.com/blog/monolithic-architecture-vs-microservices/>

Microservice Architecture: Pros vs. Cons



Advantages

- Low coupling due to isolation
- High flexibility
- High scalability
- Better error handling
- Enhanced modularity
- Failure does not impact whole system due to isolation
- Faster evolutionary iterations due to easy modification
- Avoid sinkhole problem of layered architecture and data flows

Disadvantages

- Possibly longer implementation time
- Large number of services
- Failure tendency due issues with communication between different services
- Networking issues (load balancing, latency, etc.)
- Complex testing over a distributed environment

Microservice Trade-Offs



Benefits:

- **Strong Module Boundaries:** Microservices reinforce modular structure, which is particularly important for larger teams.
- **Independent Deployment:** Simple services are easier to deploy, and since they are autonomous, are less likely to cause system failures when they go wrong.
- **Technology Diversity:** With microservices you can mix multiple languages, development frameworks and data-storage technologies.

Costs:

- **Distribution:** Distributed systems are harder to program, since remote calls are slow and are always at risk of failure.
- **Eventual Consistency:** Maintaining strong consistency is extremely difficult for a distributed system, which means everyone has to manage eventual consistency.
- **Operational Complexity:** You need a mature operations team to manage lots of services, which are being redeployed regularly.

Question

**Regarding Conway's law:
Summarize how microservices affect the workflow
within an organization.**

Distributed and Web Architectures

Intro - Modern Web Architectures by Example

Distributed Architectures

Service-Oriented Architectures

Microservice-Architectures



Cloud Computing

NIST Definition of Cloud Computing

Essential Characteristics

- 1) On-demand self-service
- 2) Broad network access
- 3) Resource pooling
- 4) Rapid elasticity
- 5) Measured service

NIST Definition of Cloud Computing

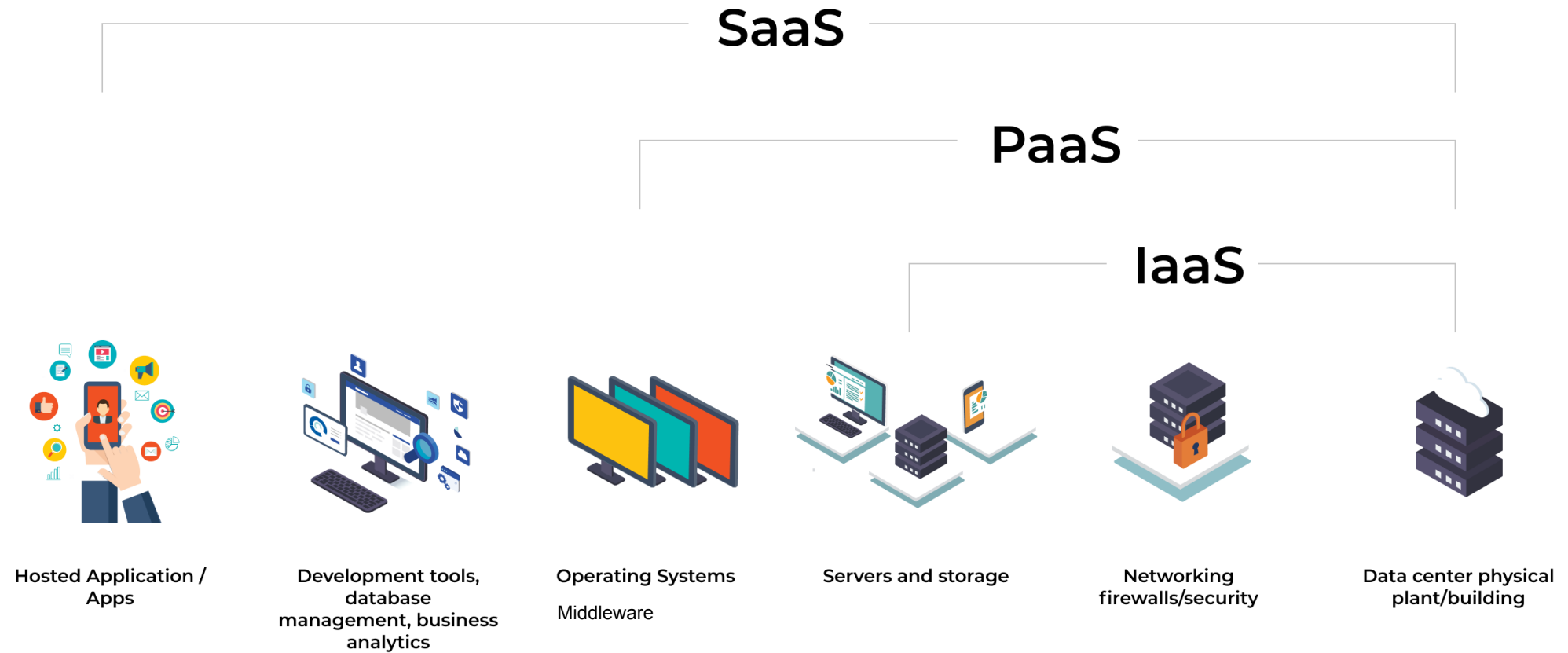
Service models

- Software-as-a-Service (SaaS)
- Platform-as-a-Service (PaaS)
- Infrastructure-as-a-Service (IaaS)

Deployment Models

- Private cloud
- Community cloud
- Public cloud
- Hybrid cloud

Cloud Service Models



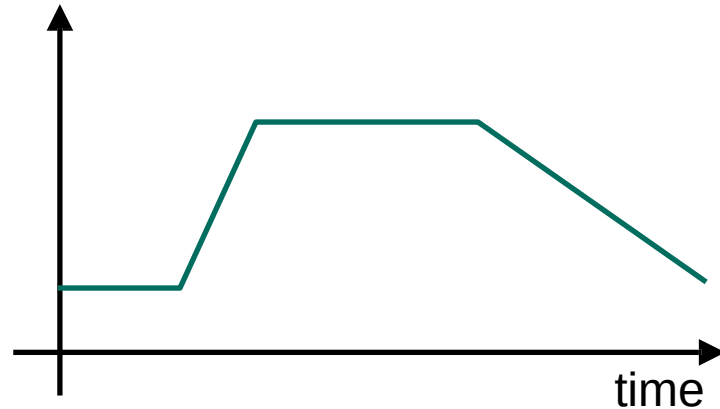
Recap: Elasticity

Definition:

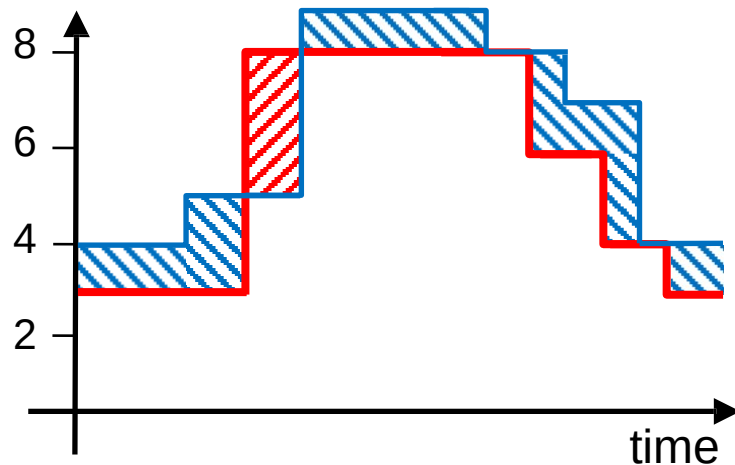
“the degree to which a system is able to adapt to workload changes [...] in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible”

Elasticity (in Cloud Computing)

Workload intensity (e.g., # requests / sec)



Amount of resources (e.g., # provisioned Virtual Machines)



Service Level Objective (SLO)

(e.g., response time ≤ 2 sec)

Resource Demand

Minimal amount of resources required to ensure SLOs



resource demand



underprovisioning



resource supply



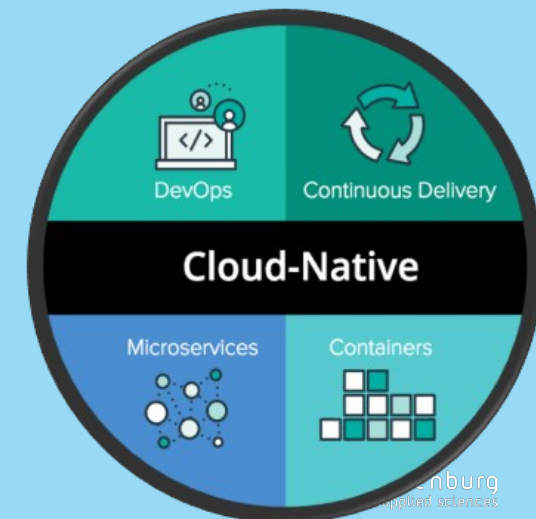
overprovisioning



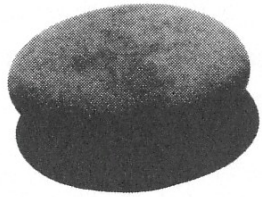
Cloud-Native Architecture

- Modern software applications are often microservice applications developed based on the DevOps paradigm and deployed in cloud environments
- A cloud-native architecture takes full advantage of cloud computing capabilities
- Usually includes
 - API-driven and **service-based architectures** like the microservice paradigm
 - Use of **DevOps** and Continuous Integration and Continuous Delivery (CI/CD) processes
 - **Container-based infrastructure**

[Red Hat, Inc. The path to cloud-native applications. 2018]



Caution: Danger of “Over-Engineering”



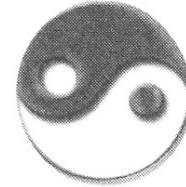
BEGINNER MIND

"I need a pattern for Hello World."



INTERMEDIATE
MIND

"Maybe I need a Singleton here."



ZEN MIND

This is a natural place for Decorator.!

*"Make everything as **simple** as possible but not simpler"*
-- Albert Einstein



Key Take Aways

- Microservices are tools that do work at high scale, but whether to use them over monolith has to be made on a case-by-case basis
- Solutions recommended by cloud providers may not be the most cost-effective, as they invariably involve using multiple costly services
- The merits of microservices versus monolithic applications are frequently overstated



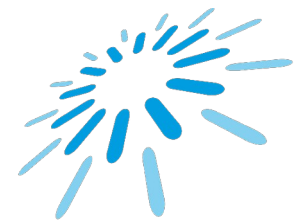
References

- Above the Clouds: A Berkeley View of Cloud Computing, TechReport, UC Berkeley, 2009.
- Cloud Programming Simplified: A Berkeley View on Serverless Computing, TechReport, UC Berkeley, 2019.
- What Serverless Computing Is and Should Become: The Next Phase of Cloud Computing, Communications of the ACM, 2021.
- The Rise of Serverless Computing, Communications of the ACM, 2019.
- Serverless Computing: One Step Forward, Two Steps Back, CIDR 2019.
- A Manifesto for Future Generation Cloud Computing: Research Directions for the Next Decade, ACM Computing Surveys, Vol. 51, No. 5, Article 105, 2018.
- Serverless is More: From PaaS to Present Cloud Computing, Internet Computing, 2018.
- Serverless End Game: Disaggregation enabling Transparency, 2020.



Until next time!

Software Architecture and Design Patterns
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences